PRESEARCH — Scuffy

> The Deep Agents insight is that **middleware** (harness layer) handles things the prompt can't enforce reliably — safety nets that fire regardless of what the LLM does. The Ralph insight is that the prompt is powerful enough for behavioral guidance if you keep it focused. The OpenCode insight is that tools should validate their inputs and give structured feedback.
>
> **The sweet spot: thin prompt + rich tools + middleware safety nets.**

Phase 1: Open Source Research

1. OpenCode (github.com/opencode-ai/opencode)

What I studied: Go-based terminal AI coding assistant. Now archived, moved to Charm's "Crush" (github.com/charmbracelet/crush). Read: `internal/llm/agent/agent.go`, `internal/llm/tools/edit.go`, `internal/llm/tools/patch.go`, `internal/diff/patch.go`, `internal/diff/diff.go`, `internal/llm/tools/` directory, README.

File editing — two strategies:

1. **Edit tool** (`tools/edit.go`) — anchor-based string replacement. Takes `file_path`, `old_string`, `new_string`. Core mechanism:
   - `strings.Index` to find the old string
   - `strings.LastIndex` to verify uniqueness — if first != last, rejects with "appears multiple times"
   - If unique, splices in the new string: `content[:index] + newString + content[index+len(oldString):]`
   - **Pre-checks:** Must read file before editing (tracks read timestamps). Checks file modtime hasn't changed since last read. Rejects stale edits.
   - **Post-checks:** Runs LSP diagnostics after every edit to catch type errors.
   - Prompt instructs LLM to include 3-5 lines of context before/after to ensure uniqueness.
2. **Patch tool** (`tools/patch.go` + `diff/patch.go`) — custom patch format for multi-file atomic changes. Format:

```
*** Begin Patch
*** Update File: path/to/file
@@ Context line (unique section identifier)
 context line
-line to remove
+line to add
 context line
*** End Patch
```

   - Context-based location: finds the `@@ section` line in the file, then matches surrounding context lines to locate the edit site
   - **Fuzzy matching fallback:** tries exact match -> trim-right whitespace -> trim-all whitespace. Tracks fuzz level, rejects patches with fuzz > 3.
   - Atomic: all files succeed or none are applied

- `PatchToCommit` -> `ApplyCommit` pipeline: parse -> validate -> transform -> write

Context management: - SQLite-backed sessions and message history - Auto-compact at 95% of model context window — summarizes conversation, creates new session with summary - File change tracking: records every version for undo capability - Separate agents: `coder` (main), `task` (subagent), `title` (generates session titles), `summarizer` (compaction)

Tool design: - bash, edit, patch, view (read file), write (full overwrite), glob, grep, ls, fetch (web), diagnostics (LSP), sourcegraph - Each tool implements `BaseTool` interface with `Info()` and `Run(ctx, ToolCall)` - Permission system: each write operation requests permission before executing - MCP server support for external tools

Error handling: - Must-read-before-edit enforced via read timestamp tracking - Mod-time staleness detection (rejects edits if file changed since last read) - Uniqueness validation on edit (rejects ambiguous matches) - LSP diagnostics after every file modification - Fuzzy match threshold on patches

What I'd take: - The edit tool's anchor-based replacement with uniqueness enforcement is clean and battle-tested. Same pattern as Claude Code's Edit tool. - Must-read-before-edit is a strong invariant — prevents blind edits. - LSP diagnostics after edits catches errors the LLM can self-correct. - The patch tool's custom format is interesting but adds complexity; the edit tool alone handles most cases.

What I'd do differently: - The patch format is proprietary — unified diff would be more portable. But the context-based matching with fuzzy fallback is smart. - Auto-compact at 95% is reactive. Better to proactively manage context budget.

---

2. Ralph (ghuntley.com/ralph/)

What I studied: Blog post by Geoffrey Huntley describing "Ralph" — an agentic loop technique built as a bash construct around Claude Code.

Architecture: - Not a framework — a **pattern**. The core is: `bash while :; do cat PROMPT.md | claude-code ; done` - Each iteration is a fresh Claude Code invocation with a focused prompt. - Monolithic process model — avoids distributed agent complexity. - "One thing per loop" — each iteration does exactly one focused task.

File editing: - Delegates entirely to Claude Code's built-in tools (Edit, Write, Bash). - No custom editing mechanism — relies on Claude Code's anchor-based replacement.

Context management — stack allocation pattern: - Each loop gets the same deterministic context: - `@fix_plan.md` — priority-sorted task list, frequently regenerated - `@specs/*` — technical specifications, one per file - `@AGENT.md` — build/test instructions, continuously updated with learnings - Context stays consistent across iterations because the same files are always loaded. - Quality degrades around 147-152k tokens (despite 200k advertised limit). - Subagents handle expensive operations (search, summarization) to preserve primary context.

Multi-agent coordination: - Up to **500 parallel subagents** for: searching, writing, analysis - Only **1 subagent** for: build/test validation (prevents backpressure collapse) - Subagents are Claude Code subagent spawns, not separate processes.

Error handling: - Failures are "deterministically bad" — reproducible and addressable via prompt tuning. - `git reset --hard` to last known good state when corruption occurs. - Fix plan acts as persistent memory — captures learnings for next iteration. - `AGENT.md` is continuously updated with discovered commands/optimizations.

Key insights: - The simplicity is the point. No framework overhead, no inter-agent protocol, no shared state beyond the filesystem. - Git is the coordination layer. Commits are checkpoints. Tags mark test-passing states. - Fresh context each loop prevents context rot. Each iteration starts clean with the same anchor documents. - "Engineers remain essential for specifying requirements, writing standards, and tuning prompts. Ralph automates execution, not architectural vision."

What I'd take: - Fresh-context-per-iteration is powerful — avoids context compaction issues entirely. - Fix plan as living document: the agent updates its own task list each iteration. - Git as the coordination and recovery mechanism. - The extreme simplicity — no framework, just a loop.

What I'd do differently: - Ralph has no observability beyond git history. We need structured tracing. - No programmatic context injection — everything is file-based. We need an API. - The 500-subagent parallelism is specific to Claude Code's architecture; we need our own orchestration model.

---

3. Open SWE / Deep Agents (github.com/langchain-ai/open-swe + langchain-ai/deepagents)

What I studied: The PRD referenced "LangChain Open Engineer" which no longer exists. Open SWE appears to be the successor/replacement — LangChain's open-source framework for building internal coding agents. It's built on **Deep Agents**, a lower-level framework that provides the actual agent loop, filesystem tools, and subagent orchestration. Read: `agent/server.py`, `agent/prompt.py`, `agent/tools/`, `agent/middleware/`, and from Deep Agents: `deepagents/graph.py`, `deepagents/middleware/filesystem.py`, `deepagents/middleware/subagents.py`, `deepagents/backends/utils.py`, `deepagents/base_prompt.md`.

Architecture — two layers:

1. **Deep Agents** (the engine) — provides:
   - Agent loop via LangGraph (`create_agent` -> compiled state graph)
   - Filesystem middleware: `ls`, `read_file`, `write_file`, `edit_file`, `glob`, `grep`
   - Subagent middleware: `task` tool for spawning child agents
   - Summarization middleware for context management
   - Sandbox backends (LangSmith, Modal, Daytona, local shell)
   - `PatchToolCallsMiddleware` for fixing malformed tool calls
   - `TodoListMiddleware` for agent planning
   - `AnthropicPromptCachingMiddleware` for cost optimization
2. **Open SWE** (the application) — adds:
   - Cloud sandbox isolation (each task in its own Linux environment)
   - Integration tools: `commit_and_open_pr`, `slack_thread_reply`, `linear_comment`, `github_comment`, `fetch_url`, `http_request`
   - Middleware: message queue injection (mid-run follow-ups), PR safety net, tool error handling
   - `AGENTS.md` context injection from repos

- Invocation surfaces: Slack, Linear, GitHub

File editing — anchor-based string replacement (same as OpenCode/Claude Code):

Core implementation in `deepagents/backends/utils.py::perform_string_replacement`:

```
def perform_string_replacement(content, old_string, new_string, replace_all=False):
    occurrences = content.count(old_string)
    if occurrences == 0:
        return f"Error: String not found in file: '{old_string}'"
    if occurrences > 1 and not replace_all:
        return f"Error: String '{old_string}' appears {occurrences} times..."
    new_content = content.replace(old_string, new_string)
    return new_content, occurrences
```

- Identical pattern to OpenCode: count occurrences, reject ambiguous matches, splice replacement
- Adds `replace_all=True` option for bulk renames (OpenCode doesn't have this)
- Must read file before editing (enforced by middleware state tracking)

Multi-agent — `task` tool with supervisor pattern: - Main agent spawns subagents via the `task` tool - Each `SubAgent` is defined declaratively: `{name, description, system_prompt, tools}` - Subagents get their own middleware stack (filesystem, todos, summarization) - A default `general-purpose` subagent is auto-added if not specified - Subagents are **ephemeral and stateless** — one invocation, one result back - Also supports `AsyncSubAgent` for background/remote tasks via LangSmith deployments - State keys like `messages`, `todos` are excluded from subagent state to prevent leakage

Context management: - Summarization middleware handles context window management - `AGENTS.md` from the repo injected into system prompt (like Claude Code's CLAUDE.md) - `check_message_queue_before_model` middleware injects follow-up messages mid-run (user can message the agent while it's working) - Large tool results are offloaded to filesystem instead of filling context - `AnthropicPromptCachingMiddleware` caches the stable prompt prefix

Middleware architecture (key insight): The middleware pattern is composable and powerful: - Each middleware wraps the agent loop and can intercept before/after model calls - `open_pr_if_needed` — safety net that ensures PR gets created even if LLM forgets - `ToolErrorMiddleware` — catches and formats tool errors - `ensure_no_empty_msg` — prevents empty messages that break the loop - `PatchToolCallsMiddleware` — fixes malformed tool calls from the LLM - Middleware is ordered: todos -> filesystem -> subagents -> summarization -> caching

Error handling: - Tool errors caught by middleware, formatted as error messages for LLM to self-correct - `PatchToolCallsMiddleware` silently fixes common LLM tool call errors - Sandbox isolation means mistakes can't affect production - `open_pr_if_needed` safety net ensures critical steps happen regardless of LLM behavior - Recursion limit of 1000 prevents infinite loops

What I'd take: - The **middleware architecture** is the standout pattern. Composable wrappers around the agent loop that handle cross-cutting concerns (error recovery, safety nets, context injection) without polluting core logic. - The `task` tool for subagents — clean, declarative, stateless spawning - `replace_all` option on edit — useful for renames - Large result eviction to filesystem — prevents context pollution - Safety net middleware (ensure PR, fix malformed tool calls)

What I'd do differently: - Deep Agents / Open SWE is Python-based and deeply coupled to

LangGraph/LangChain. We're building in TypeScript. The patterns are portable but the code isn't. - The sandbox isolation is heavy infrastructure we don't need for MVP. - The LangGraph state graph adds complexity. The core loop is just "prompt -> LLM -> tool calls -> repeat" — we can implement this directly. - 1000 recursion limit is very generous. We should cap lower and escalate.

File Editing Strategy Decision

Chosen strategy: Anchor-based replacement (string matching)

Why: - Battle-tested in all three systems studied: Claude Code, OpenCode, and Deep Agents all converge on this exact pattern — `content.count(old_string)` for validation, `content.replace(old_string, new_string)` for application. - Simpler than unified diff (no line-number fragility) and AST (no language-specific parsers needed). - The LLM naturally produces old_string/new_string pairs when prompted correctly. - Uniqueness enforcement (reject if old_string matches multiple locations) prevents wrong-site edits. - Deep Agents adds `replace_all` flag for bulk renames — we should include this.

Failure modes and mitigations:

Failure Mode	Mitigation
old_string not found (LLM hallucinates content)	Return error, LLM re-reads file and retries
old_string matches multiple locations	Reject, require more context lines for uniqueness
Stale file (modified since last read)	Track read timestamps, reject stale edits
Edit produces invalid code	Run linter/typecheck after edit, return errors for self-correction

When to fall back to full-file write:

- New file creation (no old content to match against)
- File is very small (< 10 lines) — surgical edit overhead not worth it
- Requested change touches > 50% of lines — rewrite is simpler and less error-prone

Alternative considered: Hash-based editing (Can Bölük, Feb 2026)

blog.can.ac/2026/02/12/the-harness-problem/ — reference impl: github.com/can1357/oh-my-pi

Each line gets a 2-char CRC32 hash (whitespace-stripped). The LLM references lines by hash instead of reproducing old text:

```
1:a3| function hello() {
2:f1|   return "world";
3:0e| }
```

Edits specify hash anchors + new content. The model never reproduces old text.

Evidence: Bölük benchmarked 16 models × 180 tasks × 3 runs. Results:

- Improved 15 LLMs by 5-14 percentage points. Weakest models gained up to 10x.
- ~20% fewer output tokens (no old-text reproduction).

- Immune to whitespace mismatches that plague `str_replace`. Claude Code issue #25775 documents 15-20% first-attempt failure rate on `str_replace` for tab-indented files.

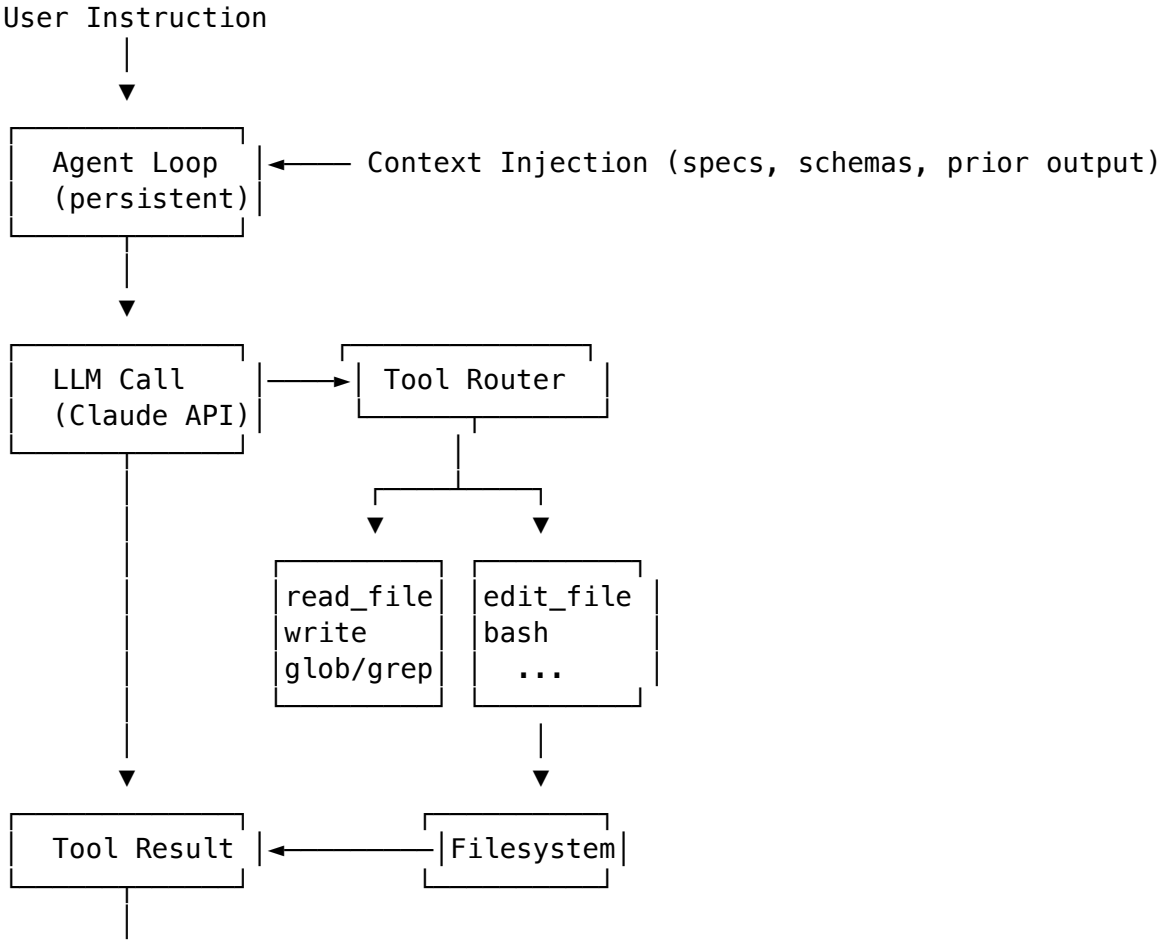
Why not for MVP:

- We're using Claude (strong frontier model) where the advantage over `str_replace` is less clear-cut — Bölük's own data showed mixed results for top-tier models.
- `str_replace` is simpler to implement, debug, and trace. Logs show old/new text directly; hash references are opaque.
- Our tool registration interface means swapping in hashline later is a small task, not an architecture change. If we see high edit failure rates post-MVP, we revisit.
- Claude Code's own team rejected hashline (NOT_PLANNED on issue #25775). They presumably have internal data on Claude's `str_replace` reliability.

Adoption status: Hive merged it (March 2026). Claude Code, Cursor, Aider, RooCode have not adopted. A Python package (`hash-edit` on PyPI) and a few smaller agents (`oh-my-openagent`) have.

Phase 2: Architecture Design

3. System Diagram



```
├──► Success -> return result to LLM -> next tool call or final answer
│
└──► Error -> return error to LLM -> LLM retries or adjusts approach
        │
        └──► Max retries exceeded -> report to user, request guidance
```

4. File Editing Strategy (detailed)

See "File Editing Strategy Decision" above (Phase 1, question 2).

Step-by-step mechanism:

1. LLM decides a file needs editing
2. LLM calls `read_file` to get current content (mandatory before edit)
3. LLM calls `edit_file` with `{file_path, old_string, new_string}`
4. Agent validates:
    a. File exists and was previously read (read-before-edit invariant)
    b. File hasn't been modified since last read (staleness check)
    c. `old_string` exists exactly once in file (uniqueness check)
5. Agent performs replacement: `content[:idx] + new_string + content[idx+len(old_string):]`
6. Agent runs linter/typecheck on modified file
7. Returns result (success + any diagnostics) to LLM

When location is wrong: - If `old_string` not found: error returned, LLM re-reads file - If multiple matches: error returned, LLM must include more context - If edit breaks types/lint: diagnostics returned, LLM self-corrects

5. Multi-Agent Design

Orchestration model: `task` tool (supervisor pattern, following Deep Agents)

All three systems converge on the same pattern: the main agent spawns subagents via a tool call. Deep Agents calls it `task`, OpenCode calls it `agent-tool`, Ralph uses Claude Code's built-in subagent spawning.

- Main agent has a `task` tool that spawns an ephemeral subagent
- Each subagent is declared with `{name, description, system_prompt, tools}`
- Subagents are **stateless** — one invocation, one result message back
- Main agent can spawn multiple subagents in parallel (parallel tool calls)
- Subagents get their own filesystem access and tool stack
- State isolation: subagent messages/todos don't leak back to parent

Communication: One-way. Main agent sends task description, subagent returns result. No back-and-forth. No inter-subagent communication.

Output merging: Main agent receives subagent results as tool messages. If multiple subagents modified the same file, main agent reviews the final file state and resolves.

Why this pattern: All three studied systems chose it independently. It's simple, deterministic, and traceable. Subagents can't interfere with each other. The main agent retains full control over sequencing and conflict resolution.

6-7. Context Injection Spec

Types of context:

Type	Format	Injection Point
Specification/requirements	Markdown string	System prompt, before first LLM call
Schema (TypeScript types, Zod)	Code string	System prompt or first user message
Prior agent output	Markdown summary	User message, before instruction
Test results	Structured text (pass/fail + stderr)	User message, after instruction
File contents	Raw text with line numbers	Tool result (read_file)

Injection point in the loop: Context is assembled into the initial messages array before the first LLM call. Dynamic context (test results, prior output) can be injected as additional user messages between turns.

8. Additional Tools

All three systems converge on a similar core toolset:

Tool	OpenCode	Deep Agents	Our Agent
Read file	`view`	`read_file`	`read_file`
Edit file	`edit`	`edit_file`	`edit_file`
Write file	`write`	`write_file`	`write_file`
Shell	`bash`	`execute`	`bash`
Find files	`glob`	`glob`	`glob`
Search content	`grep`	`grep`	`grep`
List dir	`ls`	`ls`	`list_dir`
Subagents	`agent-tool`	`task`	`task`
Planning	—	`write_todos`	`think`

Notes: - `think` is a no-op scratchpad — lets the LLM reason without side effects - Deep Agents' `write_todos` is more structured (persistent todo list). We'll start with `think` and add structured planning if needed. - All systems use `read_file` with pagination (offset/limit) for large files

---

Phase 3: Stack and Operations

9. Framework

Custom agent loop in TypeScript with middleware pattern. No LangGraph, no LangChain.

Why custom loop: The core loop is identical across all three studied systems: prompt -> LLM -> tool calls -> repeat until done. LangGraph wraps this in a state graph abstraction that adds complexity without value for our use case. Ralph proves a simple loop works. OpenCode proves it works in a production TUI. Both run a custom loop.

What we take from Deep Agents: The **middleware architecture**. Composable wrappers around the agent loop that handle cross-cutting concerns: - Error recovery (catch tool errors, format for LLM) - Context management (summarization when context grows too large) - Safety nets (ensure critical steps happen regardless of LLM behavior) - Tracing (wrap each step with timing/logging)

This gives us the extensibility of a framework without the coupling.

For multi-agent: `task` tool following the Deep Agents pattern — the main agent spawns ephemeral subagents that share the same tool stack but run in isolated context.

Tracing: Custom structured logging. Each run gets a trace ID, each step logs inputs/outputs/timing/tokens. Exportable as JSON for sharing trace links.

10. Persistent Loop

The agent runs as a **Node.js process** with a readline/stdin REPL loop:

```
while (running) {
  instruction = await readInput()     // readline or API
  context = assembleContext(instruction, injectedContext)
  result = await agentLoop(context)   // LLM <-> tools loop until done
  emit(result)
}
```

Kept alive between instructions by the Node.js event loop. No process restart needed. State (conversation history, file read timestamps) persists in memory across instructions.

11. Token Budget and Model Selection

- **Per invocation:** 4096 output tokens (sufficient for tool calls + reasoning)
- **Default model:** `claude-opus-4-6` (1M context window). Opus makes mistakes; Sonnet makes more. For a coding agent where every edit failure costs a retry loop, the cost difference is often eaten by retry tokens anyway. Opus also has the best lost-in-the-middle performance among frontier models — usable up to ~400k without significant degradation, and functional well beyond that up to 1M.
- **Model must be easily switchable** — env var or config, not hardcoded. The agent should work with any Claude model. Sonnet may be fine for subagents/workers where tasks are narrower and cheaper. Opus for the main loop.
- **Cost cliffs:**
  - Input > 400k tokens: aim to stay under this for best quality, but Opus handles up to 1M without the cliff other models hit. Summarize proactively but don't panic at 200k like you would with Sonnet or GPT.
  - Output > 8k tokens: rarely needed; if hitting this, task is too broad
  - Multi-agent fan-out: N workers × input tokens; keep N <= 5 for cost control
  - Consider Sonnet for `task` subagents to control fan-out costs

12. Bad Edit Recovery

1. **Immediate detection:** after every edit, run typecheck (`tsc --noEmit`). If errors are introduced by the edit, return them to the LLM for self-correction.
2. **Undo capability:** store file content before each edit. If the LLM can't fix the error in 2 attempts, revert to pre-edit state.
3. **Git checkpoint:** before starting a multi-step task, `git stash` or commit current state. If the entire task fails, restore.

13. Session Logging (MVP — wire up from day one)

Full session capture to JSONL, modeled on how Claude Code logs conversations. This is a middleware concern — every event flows through the harness and gets appended to a session log file. Not a tool. Not optional. Baked into the agent loop from the first commit.

Why MVP, not post-MVP:

- The PRD requires tracing with shared trace links — JSONL sessions ARE the traces
- The comparative analysis requires evidence from the Ship rebuild — logs are that evidence
- Retrofitting logging into an agent loop is painful; building it in is trivial
- Token/cost tracking for the AI Cost Analysis deliverable comes free from session logs
- Debug data for every edit failure, retry, and self-correction

What gets logged (one JSONL line per event):

```
{"type":"session_start","session_id":"ses_abc","timestamp":"...","instruction":"..."}
{"type":"llm_request","messages":[...],"model":"...","tokens_in":12500}
{"type":"llm_response","content":"...","tool_calls":[...],"tokens_out":1800,"duration_m
{"type":"tool_call","tool":"edit_file","input":{...},"call_id":"tc_1"}
{"type":"tool_result","call_id":"tc_1","output":{...},"duration_ms":8,"success":true}
{"type":"error","tool":"edit_file","message":"old_string not found","call_id":"tc_2"}
{"type":"session_end","session_id":"ses_abc","total_tokens":{"in":45000,"out":6200},"du
```

Implementation: A `LoggingMiddleware` that wraps the agent loop. Before each LLM call, log the request. After each response, log the response + tool calls. Around each tool execution, log call + result. On session start/end, log summary stats.

File location: `.scuffy/sessions/{session_id}.jsonl` — one file per session, append- only, human-readable with `cat` or `jq`.

Trace links for PRD: A trace link is just a pointer to a session JSONL file (or a hosted version). Two different sessions showing different execution paths (normal run vs. error recovery) satisfy the "two shared trace links" requirement.

14. Time Awareness (MVP — rides on session logging)

LLMs have no sense of time. They don't know the current time, how long you've been gone, or that 3 days passed between sessions. Since every JSONL event already has a timestamp, we can inject temporal context into the LLM for almost zero cost.

Three levels, all in the context assembly middleware:

1. **Current time** — inject into every LLM call's system context:

```
Current time: 2026-03-25T14:32:00-05:00 (Tue afternoon)
```

2. **Intra-session gap detection** — if significant time elapsed since the last user message in the current session (threshold: 30 min), inject a note:

   ```
   Note: 2h 15m have elapsed since the last interaction in this session.
   The user may have lost context — reorient briefly before continuing.
   ```

   This prevents the agent from continuing mid-thought as if the user just stepped away for 5 seconds when they actually went to lunch.

3. **Inter-session context** — on session start, read the previous session's last event timestamp and inject:

   ```
   Last session ended 3 days ago (2026-03-22T23:41:00).
   Session summary: [last session's final instruction + outcome]
   ```

   The agent starts with temporal orientation instead of the Alzheimer's-relative experience of every conversation starting from a void.

Implementation: This is context injection in the middleware that assembles messages before each LLM call. No new tools, no new logging — just reading timestamps that are already in the JSONL and prepending a line to the system prompt.

15. Trace Format (per-step summary view)

The JSONL session log is the raw data. For the CODEAGENT.md trace links and human review, we can derive a summary view:

```
{
  "trace_id": "tr_abc123",
  "instruction": "Add error handling to the login function",
  "steps": [
    {
      "step": 1,
      "tool": "read_file",
      "input": {"path": "src/auth/login.ts"},
      "output": {"lines": 45, "truncated": false},
      "duration_ms": 12
    },
    {
      "step": 2,
      "tool": "edit_file",
      "input": {
        "file_path": "src/auth/login.ts",
        "old_string": "const result = await db.query(...)",
        "new_string": "const result = await db.query(...).catch(..."
      },
      "output": {"status": "success", "diagnostics": []},
      "duration_ms": 8
    },
    {
```

```
      "step": 3,
      "tool": "bash",
      "input": {"command": "npx tsc --noEmit"},
      "output": {"exit_code": 0, "stdout": "", "stderr": ""},
      "duration_ms": 3200
    }
  ],
  "total_duration_ms": 15420,
  "tokens": {"input": 12500, "output": 1800},
  "status": "success"
}
```

Phase 4: Tool Design Philosophy

Tools as Prompt Compression

Every time the agent calls `bash` with a domain-specific command, you're paying tokens for the LLM to reconstruct CLI syntax it already learned once. A typed tool amortizes that cost to zero — the tool schema IS the documentation. The LLM only needs to know *when* to use a tool, not *how* to spell the CLI.

Compare beads via bash vs. a first-class tool:

Bash approach (current):

- Prompt must teach `br` syntax (many lines of examples and flags)
- LLM constructs command strings — typos, wrong flags, missing quotes
- Output is CLI text the LLM must parse
- Every fresh agent re-learns the syntax
- Errors are opaque strings

First-class tool approach:

```
// The schema teaches the LLM the interface. No CLI syntax needed.
beads_create({
  title: "Add error handling to auth",  // required string
  type: "task",                         // enum: task | bug | chore
  priority: 1,                          // 0-3
  description: "...",                   // required string
  labels: ["mvp"],                      // optional array
})
// Returns: { id: "sc-a1b2", title: "...", status: "created" }
```

The tool description is *smaller* than the CLI documentation. Validation catches bad inputs before execution. Output is structured JSON. Tracing is automatic.

When to Make a Tool vs. Use Bash

Make it a tool	Keep it as bash
Agent uses it frequently (beads, typecheck)	One-off or rare operation
Complex interface where LLM makes syntax errors	Simple, well-known CLI (`git status`)
You want structured input/output	You need shell composition (pipes, redirects)
Operation has domain semantics worth encoding	Thin wrapper with no added value
You want harness-level control/tracing	Full flexibility matters more than structure

Domain Tools for This Agent

This is not a general-purpose coding agent. It's a personal agent for specific workflows: coding, issue tracking, project management via beads, TypeScript development. The tool set should reflect that.

High value (frequent use, complex interface, structured I/O matters):

1. **`beads`** — wraps `br` with typed sub-operations (create, update, list, ready, dep-add, label-add). Used in every Tower and Trench session. Complex CLI with many subcommands. Structured JSON output is dramatically more useful than CLI text — the agent can reliably check dependencies, find ready tasks, update status without fragile string parsing.
2. **`typecheck`** — wraps `tsc --noEmit` with parsed output. Returns structured errors: `{file, line, column, message, code}[]`. The LLM can then precisely target fixes instead of parsing compiler output. Runs after nearly every edit.
3. **`project_context`** — reads CLAUDE.md + ARCHITECTURE.md + CONTEXT.md + SCOREBOARD.md in one call. Every agent session starts with 3-5 `read_file` calls for the same files. One tool call, one structured result, fewer tokens.

Medium value (simplifies common operations):

4. **`test`** — wraps test execution with structured pass/fail/skip counts and failing test details. Saves the LLM from parsing test runner output.
5. **`lint`** — wraps eslint/prettier with structured diagnostics. Same pattern as typecheck.

Low value now, high value for capstone:

6. **`evaluate_change`** — analyze a set of file changes as a graph (dependencies, impact, risk). This is capstone territory but the tool interface should be ready for it.

The Architectural Decision That Matters

The specific tools are less important than the **tool registration interface**. If adding a tool is trivial:

```
const beadsTool: Tool = {
  name: "beads",
  description: "Manage project issues via beads (br)",
  parameters: BeadsParamsSchema,  // Zod schema
  execute: async (params) => { /* ... */ },
```

```
};

agent.registerTool(beadsTool);
```

…then "should we add a beads tool?" becomes a 30-minute implementation task, not an architectural decision. The harness design makes or breaks extensibility. The specific tools are fill-in-the-blank.

Where Intelligence Lives

There are three places to put smarts in an agent system:

Layer	What goes here	Example
Prompt	Behavioral guidance, domain knowledge, task framing	"Read files before editing. Use imperative commit messages."
Tools	Capabilities with typed interfaces and structured I/O	`edit_file`, `beads`, `typecheck`
Harness (middleware)	Cross-cutting concerns, safety nets, enforcement	Error recovery, context summarization, tracing

The prompt is for guidance the LLM can follow *most of the time*. Tools are for capabilities with clear input/output contracts. Middleware is for invariants that must hold *regardless of what the LLM does* — safety nets, not suggestions.

Scope Management

Build for Shipyard MVP:

- Clean tool registration interface (Zod schema -> typed tool -> register)
- Core tools: read_file, edit_file, write_file, bash, glob, grep, list_dir, task, think
- Middleware: error recovery, context management
- **Session logging middleware (JSONL) — from the first commit.** This is not optional and not post-MVP. It satisfies the tracing requirement, generates comparative analysis evidence, and provides cost tracking data. Wire it up before anything else.
- Maybe one domain tool (beads or typecheck) as proof of extensibility

Build post-MVP but design the interface for now:

- beads tool, typecheck tool, project_context tool
- These plug into the interface trivially if the interface is right

Defer to capstone but keep the door open:

- Graph evaluation tools, change impact analysis
- The tool interface from Shipyard becomes the foundation

The interface costs almost nothing to get right. Domain tools are easy to add later if the interface is clean. Trying to build everything for MVP is how you miss Tuesday.

Why Bring Domain Tools Into Your Personal Codebase

- **Your workflows are repetitive.** Beads in every session. Same project files. Same validation commands. Each is a repeated token tax and error opportunity.
- **Your agent isn't general-purpose.** It needs to be great at YOUR workflows, not everything. A beads tool that knows your issue tracking semantics beats generic bash.
- **Tools compound.** Structured beads output means the agent can reliably reason about dependencies, blocking tasks, readiness — without fragile string parsing.
- **Tools are self-documenting.** In 6 months, the tool definitions tell you exactly what your agent can do. A prompt blob is harder to audit.

Why You Might Not

- **Maintenance.** Every tool is code you maintain. CLI changes require tool updates.
- **Premature abstraction.** If your workflow is still evolving, tools lock you in.
- **Scope.** For Shipyard specifically, domain tools add risk to a tight timeline.

The call: **design the interface now, implement domain tools post-MVP.** The interface is the high-leverage decision. The specific tools are fill-in-the-blank afterward.